

Software Requirements Specifications

Version: 3.0

Last Updated: 04 September 2026



Prepared by



Introduction	2
User Stories / User Characteristics	3
High-level Use Case Diagram	5
Use Cases	5
Authorisation	5
Log Time (Developer)	6
Timesheets (Developer)	7
Task Management (Developer)	8
Project Visibility (Developer)	9
Project & Team Oversight (Manager)	10
Organisation Oversight (Admin)	11
Timesheet Approval (Manager)	12
Functional Requirements	13
FR1. Authentication & Session Management	13
FR2. Time Logging	13
FR3. Timesheets	14
FR4. Task Management	14
FR5. Project Visibility & Oversight	15
Non-functional requirements	15
1. Performance	15
2. Scalability	16
3. Security	17
4. Availability and Reliability	17
5. Maintainability	18
Domain Model	19

Introduction

Traditional time-tracking tools act as passive digital notebooks, leaving developers, managers, and admins to manually reconcile hours logged against project budgets and deadlines. Without a shared, real-time view of who is working on what, teams will frequently discover that a project is over budget or behind schedule only after the fact, once a timesheet has already been approved and the week is closed. Momently bridges the gap between raw time entries and actionable project intelligence; it gives Developers a faster way to log and submit their hours, and gives Managers and Admins a live, role-scoped view of project progress, team hours, and budget consumption, without waiting on a manual report. The system is role-aware from the ground up, the same screen shows a Developer their own assigned work, a Manager their whole workspace, and an Admin the entire organisation, without separate applications.

User Stories / User Characteristics

User Management & Authentication

- As a user, I want to enter my company email and password and hit submit so that the system authenticates me and grants access to my role-appropriate dashboard.
- As a user, I want to fill out the registration form with my name, company email, and password so that the system creates my new account and I am assigned the default Developer role.
- As a user, I want to verify my email address after registration so that I can unlock full access to the system.
- As a user, I want to select the log-out option so that the system securely ends my active session and returns me to the login screen.

Workspace & Role Management

Manager

- As a manager, I want to view my workspace so that I can see all projects and members in my team.
- As a manager, I want to invite a user to my workspace so that I can add new team members.
- As a manager, I want to transfer workspace ownership to another user so that I can hand over responsibility when needed.

- As a manager, I want to assign Project Manager roles to any workspace member so that they can have project permissions for a specific project

Administrator

- As an admin, I want to view all workspaces in the system so that I have full visibility over the entire application.
- As an admin, I want to delete a workspace so that I can remove it from the system when it is no longer needed.
- As an admin, I want to invite a user to any workspace so that I can add members to any team or organisation.
- As an admin, I want to change the owner of a workspace so that I can transfer ownership to another user.

Timesheets

Developer

- As a developer, I want to view my current week's time entries grouped by task so that I can see my daily and weekly totals at a glance.
- As a developer, I want to filter my timesheets by status (Draft, Submitted, Approved, Rejected) so that I can quickly find the ones I still need to act on.
- As a developer, I want to submit my week's draft entries as a single timesheet so that my manager can review and approve them.
- As a developer, I want to see my manager's rejection reason inline on a rejected timesheet so that I know exactly what to correct before resubmitting.

Manager

- As a manager, I want to view all submitted timesheets in my workspace so that I can view them efficiently
- As a manager, I want to approve a submitted timesheet so that it becomes locked in as finalised
- As a manager, I want to reject a submitted timesheet with a written reason so that the developer knows exactly what to fix.

Administrator

- As an admin, I want to view all timesheets across all workspaces so that I can monitor the overall submission and approval status of the organisation.
- As an admin, I want to approve any timesheet in any workspace so that I can finalise timesheets when needed.
- As an admin, I want to reject any timesheet with a reason so that I can provide feedback on submissions.

Task Management

Developer

- As a developer, I want to view all tasks assigned to me so that I know what work I still need to complete.
- As a developer, I want to filter and search my tasks by status, priority, or title so that I can focus on what matters right now.
- As a developer, I want to update a task's status so that my current progress is reflected on screen.

Manager

- As a manager, I want to view all tasks within a project so that I can see what work is in progress.
- As a manager, I want to create tasks and assign them to team members so that work is properly distributed.
- As a manager, I want to set task priorities and due dates so that the team knows what to focus on first.

Project Management

Developer

- As a developer, I want to view the projects I am personally assigned to so that I know which projects I can log time against.
- As a developer, I want to see my own hours logged and each project's progress so that I understand how my work contributes to the whole.

Manager

- As a manager, I want to view every project in my workspace, not just the ones I'm personally on, so that I have full visibility over my team's workload.
- As a manager, I want to see each project's team roster and each member's hours logged so that I can spot over- or under-loaded team members.
- As a manager, I want to see a project's progress against its budgeted hours so that I can tell if a project is on track.
- As a manager, I want to see each project's budgeted cost and hourly rate so that I can manage cost, not just time.

Administrator

- As an admin, I want to view every project across every workspace in the organisation, not just my own, so that I have a single, org-wide view of all work in flight.

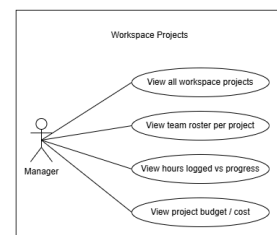
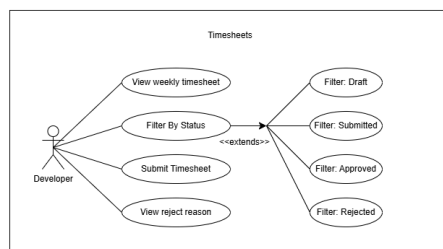
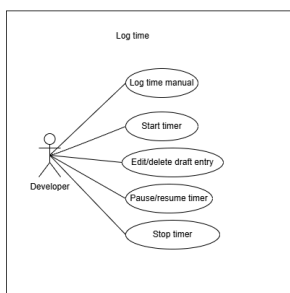
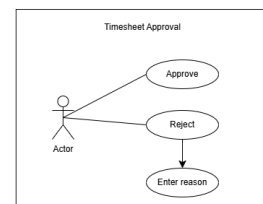
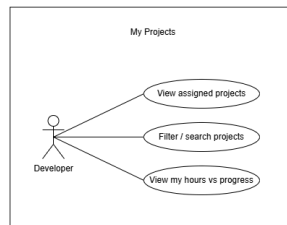
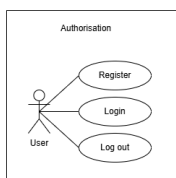
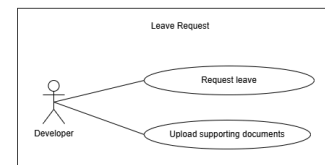
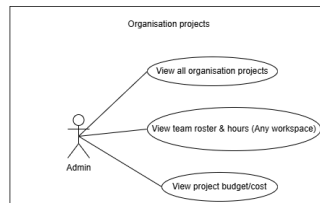
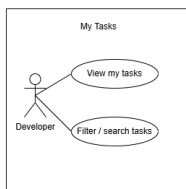
- As an admin, I want to see the same team roster, hours, progress, and cost detail as a manager, but for any project in the organisation, so that I can support any team without needing to be added to it.

Leave Request

Developer

- As a developer, I want to request for leave so that my timesheets, time entries don't affect my insights against being flagged as a non-worker.
- As a developer, I want to upload supporting documents so my manager can approve my request.

High-level Use Case Diagram

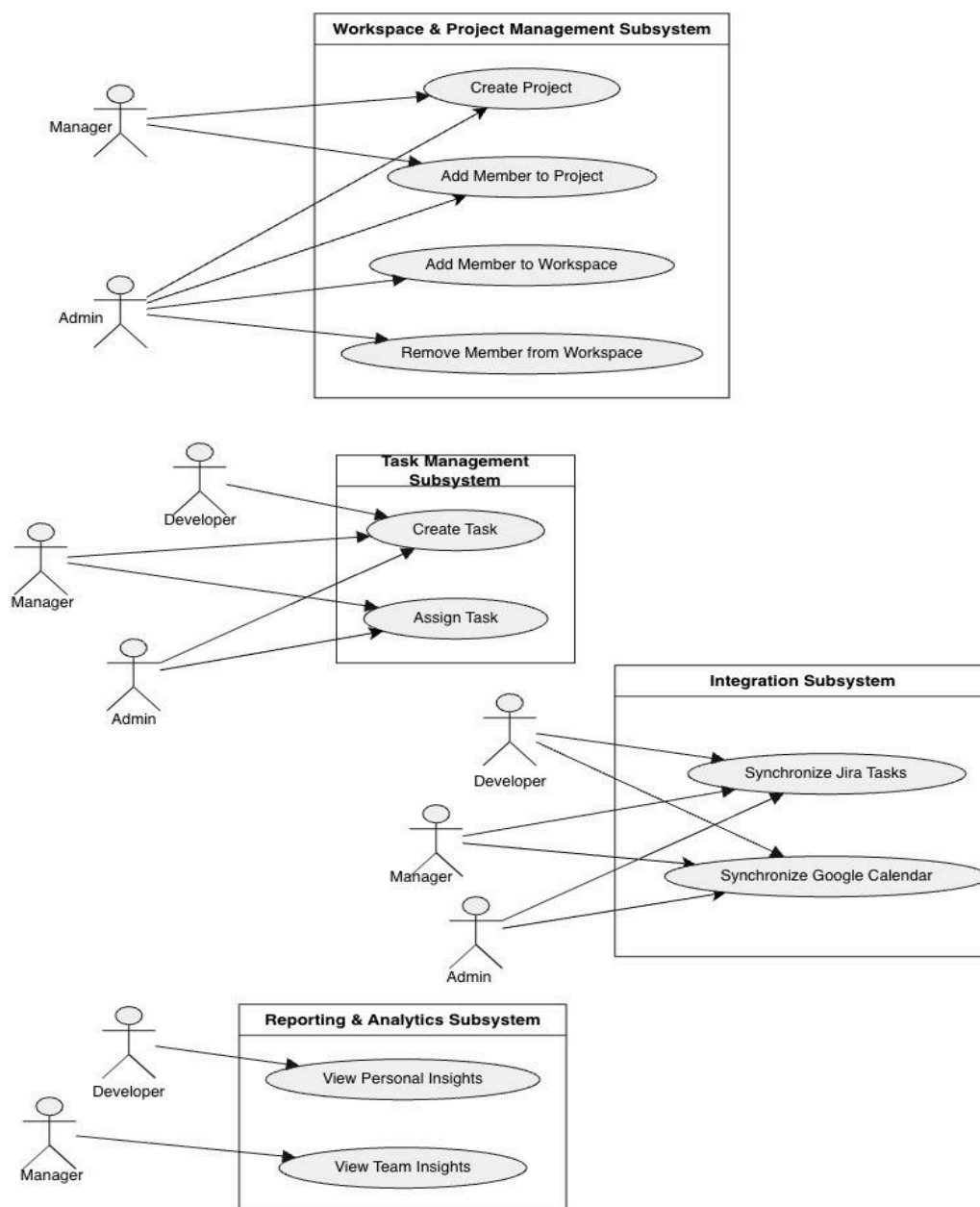


Use Cases

Each use case below states **TUCBW** (This Use Case Begins When) and **TUCEW** (This Use Case Ends When) as recommended by the lecture slides and textbook.

Note: A Manager inherits developer as a Manager can be a developer in a different project

Authorisation



Register

- TUCBW the user, not yet authenticated, submits the registration form with first name, last name, company email, and password.
- TUCEW the system creates the new account (default role: Developer) and the user is authenticated or routed to log in with their new credentials.

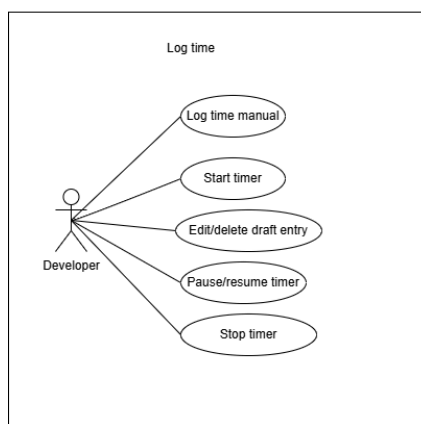
Login

- TUCBW the user enters their company email and password and hits submit.
- TUCEW the system authenticates the credentials, issues a JWT, and grants access to the user's role-appropriate, guarded routes.

Log out

- TUCBW the user selects the log-out option from the app menu.
- TUCEW the system clears the active session/token and returns the user to the login screen.

Log Time (Developer)



Log time (manual)

- TUCBW the user opens the Log Time screen and opens the manual-entry panel.
- TUCEW the new time entry is saved as a draft and appears in the current day's list.

Start timer

- TUCBW the user selects a project and task and clicks "Start Timer."
- TUCEW an active timer session begins, and elapsed time displays and updates every second.

Pause / resume timer

- TUCBW the user pauses an active timer.
- TUCEW elapsed time is held exactly, and resuming continues counting from that point without loss.

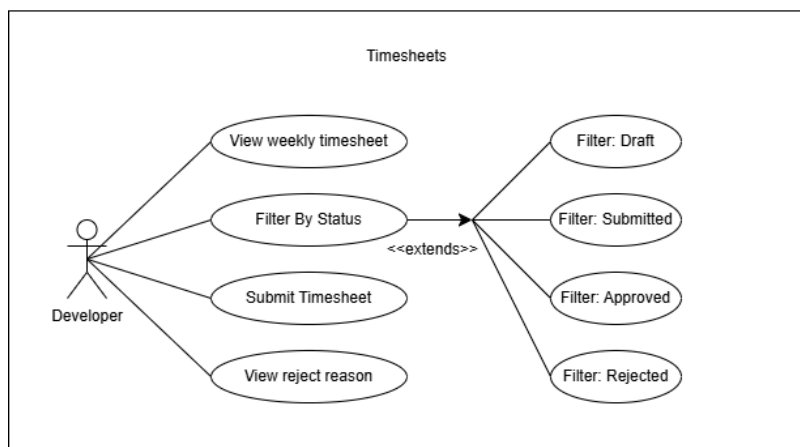
Stop timer

- TUCBW the user clicks "Stop" on an active timer.
- TUCEW a draft time entry is created automatically from the elapsed duration, with no further user action required.

Edit / delete draft entry

- TUCBW the user selects a draft entry and chooses to edit or delete it.
- TUCEW the entry's changes are saved, or the entry is removed, provided its parent timesheet is still in Draft status.

Timesheets (Developer)



View weekly timesheet

- TUCBW the user navigates to the Timesheets screen.
- TUCEW the system displays the current week's entries grouped by task, with daily totals, a grand total, and the timesheet's approval status.

Filter by status

- TUCBW the user selects a status filter (Draft, Submitted, Approved, or Rejected).
- TUCEW the system re-queries and displays only timesheets matching that status.

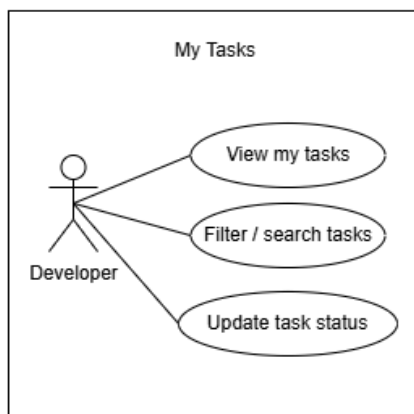
Submit timesheet

- TUCBW the user, viewing a Draft and unlocked timesheet, clicks "Submit Timesheet" and confirms.
- TUCEW the timesheet's status changes to Submitted, the week is locked from further editing, and a success confirmation is shown.

View rejection reason

- TUCBW the user opens a timesheet in Rejected status.
- TUCEW the system displays the manager's written rejection reason inline alongside the entries.

Task Management (Developer)



View my tasks

- TUCBW the user navigates to the My Tasks screen.
- TUCEW the system displays all tasks assigned to the user, with live counts by status.

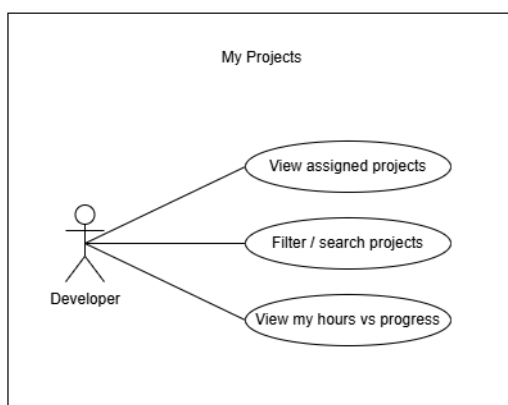
Filter / search tasks

- TUCBW the user applies a status filter or types a search term.
- TUCEW the displayed task list updates to match the filter or search term.

Update task status

- TUCBW the user changes a task's status from the status dropdown.
- TUCEW the task's status updates on screen to reflect the new value.

Project Visibility (Developer)



View assigned projects

- TUCBW the user navigates to the Projects screen.
- TUCEW the system displays every project the user is personally a member of, as cards showing name and status.

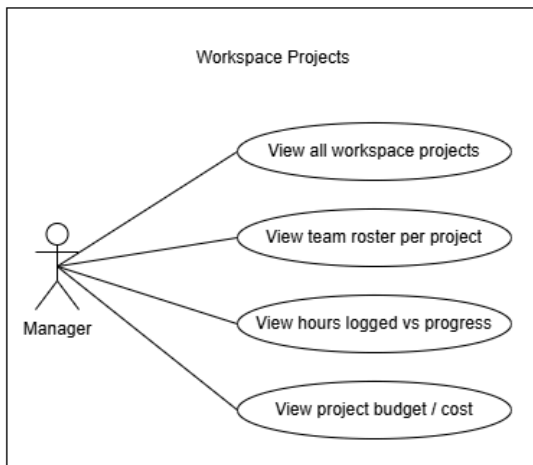
Filter / search projects

- TUCBW the user applies a status filter or types a project name into search.
- TUCEW the displayed project cards update to match the filter or search term.

View my hours vs. progress

- TUCBW a project card's detail call resolves after the initial list loads.
- TUCEW the card displays the user's own hours logged on that project, the project's overall progress percentage, and the initials of its team members.

Project & Team Oversight (Manager)



View all workspace projects

- TUCBW a user with the Manager role navigates to the Projects screen.
- TUCEW the system displays every non-deleted project in the manager's workspace, regardless of whether the manager is personally assigned to it.

View team roster per project

- TUCBW a project card's detail call resolves for a manager.
- TUCEW the system displays every team member on that project alongside their individually logged hours.

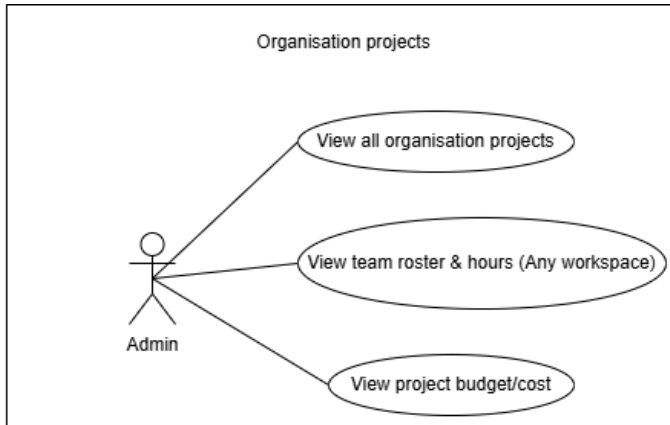
View hours logged vs. progress

- TUCBW a project's detailed data.
- TUCEW the system displays total hours logged for the project and a progress bar (clamped to 100%) representing hours logged against budgeted hours.

View project budget / cost

- TUCBW a manager views a project card or its details.
- TUCEW the system displays that project's budgeted hours, hourly rate, and total cost, which are hidden from a Developer viewing the same project.

Organisation Oversight (Admin)



View all organisation projects

- TUCBW a user with the Admin role navigates to the Projects screen.
- TUCEW the system displays every non-deleted project across every workspace in the organisation.

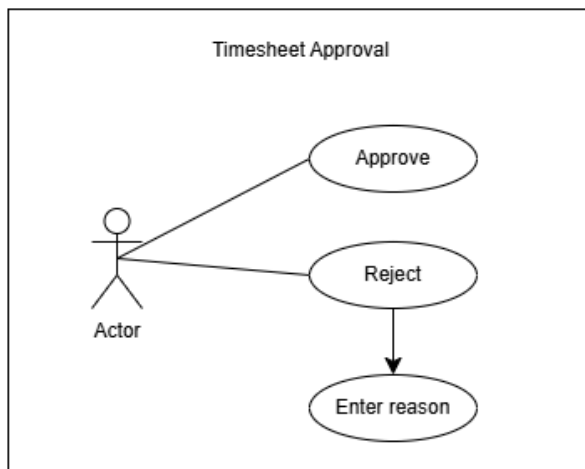
View team roster & hours (any workspace)

- TUCBW an admin views the details of any project, regardless of workspace.
- TUCEW the system displays that project's team roster and each member's hours logged, identically to the manager view.

View project budget / cost

- TUCBW an admin views any project card or its details.
- TUCEW the system displays that project's budgeted hours, hourly rate, and total cost.

Timesheet Approval (Manager)



Note: For this, no frontend ui exists; backend exists and is scheduled to Demo 3.

Approve

- TUCBW a manager, viewing a Submitted timesheet, clicks "Approve" and confirms.
- TUCEW the timesheet's status changes to Approved and is locked, and a success confirmation is shown.

Reject

- TUCBW a manager, viewing a Submitted timesheet, clicks "Reject."
- TUCEW, after the manager enters a reason and confirms, the timesheet's status changes to Rejected with that reason stored and visible to the developer.

Enter reason

- TUCBW the manager opens the reject dialog.
- TUCEW a non-empty reason is captured and attached to the rejection before the request is sent.

AI Insights & GitHub Integration (Developer/Manager)

View personal insights (Developer)

TUCBW the user, in the Developer role, navigates to the Insights screen.

TUCEW the system displays the user's weekly summary, productivity score,

task-switching rate, delivery forecast, and any flagged anomalies for their own logged time.

View team insights (Manager)

TUCBW the user, in the Manager role, navigates to the Insights screen.

TUCEW the system displays workspace-wide burnout risk flags, delivery forecasts per project, and team productivity trends across all members.

Switch insights scope

TUCBW the user selects "Overall" or "By project" from the scope switcher on the Insights screen.

TUCEW the displayed insight cards and charts update to reflect only the selected scope.

Connect GitHub account

TUCBW the user selects "Connect GitHub" from the Insights screen, having no active GitHub connection.

TUCEW the user is redirected to GitHub's OAuth consent screen, and upon authorisation, the system stores the connection and marks the user as GitHub-connected.

Sync GitHub activity

TUCBW a GitHub-connected user selects "Sync" on the Insights screen.

TUCEW the system fetches the user's recent commit activity and refreshes the correlation between logged hours and GitHub contributions.

View hours-vs-commits comparison

TUCBW a GitHub-connected user views the "Git commits vs effort" insight.

TUCEW the system displays a chart comparing logged hours against commit activity for the selected period, along with a correlation note.

Workspace & Role Management

Workspace membership

- The system will allow a Manager or Admin to invite a user to a workspace by email.
- The system will assign the Developer role to an invited user by default upon joining a workspace.
- The system will allow a Manager to assign Project Manager permissions to any workspace member, scoped to a specific project only.

- The system will allow a Manager or Admin to transfer ownership of a workspace to another existing member.
- The system will allow an Admin to view all workspaces in the system, regardless of the admin's own membership.
- The system will allow an Admin to delete a workspace, removing its associated data from the system.
- The system will restrict workspace-management actions (invite, role assignment, ownership transfer, deletion) to users holding the Manager or Admin role only.

Leave Requests

Leave request submission

- The system will allow a Developer to submit a leave request specifying a start date, end date, and reason.
- The system will allow a Developer to attach a supporting document to a leave request.
- The system will save a new leave request with a status of Pending.
- The system will exclude time within an approved leave period from being flagged as non-working time in a developer's insights or timesheets.

Leave request review

- The system will allow a Manager to view all Pending leave requests within their workspace.
- The system will allow a Manager to approve a Pending leave request.
- The system will allow a Manager to reject a Pending leave request with a written reason.
- The system will restrict the Approve/Reject controls for leave requests to users holding the Manager role only.

AI Insights & GitHub Integration

Personal insights (Developer)

- The system will generate and display a Developer's weekly summary, productivity score, task-switching rate, and delivery forecast based on their own logged time.
- The system will flag anomalous time entries (e.g. unusually long single-day durations) for the Developer's review.
- The system will allow a Developer to switch between an "Overall" and "By project" scope when viewing their insights. Team insights (Manager)

- The system will generate and display workspace-wide burnout risk indicators, per-project delivery forecasts, and team productivity trends for a Manager.
- The system will allow a Manager to switch between an "Overall" and "By project" scope when viewing team insights.
- The system will restrict team-scoped insights to users holding the Manager or Admin role only. GitHub integration
- The system will allow a user to connect their GitHub account via OAuth.
- The system will allow a connected user to manually trigger a sync of their recent GitHub commit activity.
- The system will calculate and display a correlation between a user's logged hours and their GitHub commit activity once connected.
- The system will display a note prompting the user to connect GitHub when no connection exists, in place of GitHub-derived insights.

Functional Requirements

FR1. Authentication & Session Management

FR1.1. Account access

- FR1.1.1. The system will allow a user to register a new account using a company email and password.
- FR1.1.2. The system will assign the Developer role to a newly registered account by default.
- FR1.1.3. The system will allow a user to log in with their company email and password and receive a signed session token (JWT).
- FR1.1.4. The system will attach the session token automatically to every subsequent authenticated request.
- FR1.1.5. The system will redirect an unauthenticated user attempting to access a guarded route to the login screen.
- FR1.1.6. The system will allow a user to log out, ending their active session

FR2. Time Logging

FR2.1. Manual time entry

- FR2.1.1. The system will allow a developer to select a project from only the projects they are assigned to.
- FR2.1.2. The system will allow a developer to select a task belonging only to the currently selected project.
- FR2.1.3. The system will calculate entry duration from the entered start and end time; duration will never be entered manually.
- FR2.1.4. The system will save a new manual entry with a status of Draft.
- FR2.1.5. The system will allow a developer to edit or delete a draft entry, provided its timesheet is not yet submitted or locked.

FR2.2. Timer-based time entry

- FR2.2.1. The system will allow a developer to start a timer against a selected project and task.
- FR2.2.2. The system will allow only one active timer per developer at any time.
- FR2.2.3. The system will allow a developer to pause and resume an active timer without loss of elapsed time.
- FR2.2.4. The system will automatically create a Draft time entry when a timer is stopped, requiring no further action from the developer.

FR3. Timesheets

FR3.1. Timesheet review

- FR3.1.1. The system will display a developer's current week's time entries grouped by task, with daily and weekly totals.
- FR3.1.2. The system will allow a developer to filter their timesheets by status (Draft, Submitted, Approved, Rejected).
- FR3.1.3. The system will display a manager's rejection reason inline on a rejected timesheet.

FR3.2. Timesheet submission

- FR3.2.1. The system will allow a developer to submit a Draft, unlocked timesheet for approval.
- FR3.2.2. The system will lock a timesheet from further editing immediately upon submission.
- FR3.2.3. The system will allow a rejected timesheet to be corrected and resubmitted.

FR3.3. Timesheet approval (Manager)

- FR3.3.1. The system will allow a user with the Manager role to approve a Submitted timesheet.
- FR3.3.2. The system will allow a user with the Manager role to reject a Submitted timesheet, requiring a non-empty written reason.

- FR3.3.3. The system will lock a timesheet once it has been approved or rejected.
- FR3.3.4. The system will restrict the Approve/Reject controls to users holding the Manager role only.

FR4. Task Management

FR4.1. Personal task list

- FR4.1.1. The system will display all tasks assigned to the current user.
- FR4.1.2. The system will allow the user to filter tasks by status (To Do, In Progress, Done, Blocked).
- FR4.1.3. The system will allow the user to search tasks by title.
- FR4.1.4. The system will display live counts of active, completed, and blocked tasks.
- FR4.1.5. The system will allow the user to change a task's status.

FR5. Project Visibility & Oversight

FR5.1. Role-scoped project listing

- FR5.1.1. The system will display, to a Developer, only the projects that developer is personally assigned to.
- FR5.1.2. The system will display, to a Manager, every non-deleted project within that manager's workspace, regardless of personal assignment.
- FR5.1.3. The system will display, to an Admin, every non-deleted project across every workspace in the organisation.
- FR5.1.4. The system will allow the user to filter the displayed project list by status and search by project name.

FR5.2. Project detail

- FR5.2.1. The system will display, for a selected project, its full team roster and each member's individually logged hours.
- FR5.2.2. The system will display total hours logged for a project and a progress percentage against its budgeted hours, clamped between 0% and 100% for display.
- FR5.2.3. The system will display budgeted hours, hourly rate, and total cost for a project only to users holding the Manager or Admin role.
- FR5.2.4. The system will deny project detail access to a user with no membership on that project and no Manager/Admin role.

Non-functional requirements

1. Performance

The system must be able to handle time-critical operations such as starting/stopping timers, submitting timesheets, viewing dashboards, with minimal delay. Users should perceive the system as responsive, even during peak times.

Quantification:

Under normal operating conditions, 95% of API requests must complete within 2 seconds. The system must support at least 100 concurrent users while maintaining the specified response-time targets.

Measurement:

- Apache JMeter for load and performance testing
- Spring Boot Actuator for application metrics
- Prometheus for metrics collection
- AWS CloudWatch for infrastructure monitoring

Example test:

Performance test to simulate 100 concurrent users starting and stopping timers, viewing timesheets, submitting timesheets, loading dashboards and retrieving projects and tasks. The test passes if at least 95% of requests complete within 2 seconds and the error rate remains below 1%.

2. Scalability

The system must be designed to handle growth in users, projects, and data volume. While the initial implementation will include Jira, GitHub, Google Calendar integrations and AI-driven predictive features, the architecture must also support the addition of future external integrations such as Microsoft Teams or a different Single Sign On service without requiring a complete system redesign.

Quantification:

The system must support a 200% increase in workload from the initial baseline while maintaining at least 90% of the original performance.

Measurement:

- Apache JMeter by progressively increasing the traffic
- Prometheus for metrics collection
- AWS CloudWatch for monitoring CPU, memory, database and infrastructure scaling

Example test:

The system is tested with 100 users as the baseline performance. Then tested with 300 users and have a less than 10% performance degradation from the baseline.

3. Security

The system must be able to handle sensitive employee data: timesheets, productivity scores, cost information, and personal user details. Security must protect this data from unauthorized access, ensure accountability for all changes, and comply with relevant privacy regulations (POPIA). Authentication mechanisms must be extensible to support future SSO providers.

Quantification:

- 100% of API endpoints containing protected data must require authentication, except explicitly public endpoints
- 100% of authorisation decisions must be enforced server-side using Spring Security
- 100% of administrative accounts must support and enforce MFA
- 0 critical or high-severity vulnerabilities may remain unresolved before a production release
- 100% of security-sensitive actions, including timesheet approvals, rejections, role changes, and access changes, must be recorded in the audit log

Measurement:

- Spring Security for authentication and authorisation
- JWT for stateless authentication
- AWS Parameter Store for secure configuration and secrets
- PostgreSQL audit tables for auditability
- Administrative accounts must not be able to access administrative functionality without successfully completing MFA

Example test:

Backend engineers should test:

- Roles cannot access endpoints they do not have access to
- User cannot access another workspaces' data
- Expired JWT tokens are rejected
- Invalid or revoked tokens are rejected
- Sensitive actions generate audit records

4. Availability and Reliability

The system must be reliably available for users to log their time, submit timesheets, and for managers to approve them. Business decisions depend on accurate and timely timesheet data, so downtime must be minimized. In the event of a failure, the system must recover and restore full functionality within a defined timeframe.

Quantification:

The system must:

- Achieve at least 99.9% monthly availability, excluding scheduled maintenance
- Restore full service after a failure without losing committed time entries or approved timesheets
- Recover from a critical application failure within 2 hours
- Maintain a minimum 1% data-loss rate for committed database transactions

Measurement:

- AWS CloudWatch for availability and infrastructure monitoring
- Spring Boot Actuator for health endpoints
- Prometheus for uptime and health metrics

Example test:

A staging test deliberately stops the Spring Boot application container.

The system must:

1. Detect that the application is unhealthy
2. Restart or replace the failed instance
3. Restore service within 5 minutes
4. Preserve all previously committed time entries
5. Prevent corrupted or partially completed timesheet submissions

5. Maintainability

The system must be easy to extend, and test. As new features are added (additional external integrations, cost management, business owner features), the codebase must remain understandable and modifiable without breaking existing functionality. The architecture must support adding new external SSO providers or switching between them with minimal effort.

Quantification:

- At least 80% automated test coverage for backend business logic
- At least 70% automated test coverage for frontend business logic
- Adding a new external integration must require changes primarily within the relevant integration adapter package
- At least 90% of existing automated tests must continue to pass after the addition of a new integration
- New features must not introduce new critical or high-severity code quality issues

Measurement:

- JUnit 5
- Mockito
- SpringBoot Test
- Jest
- JaCoCo for Java code coverage

Example test:

Architecture should enforce that business logic must not directly depend on:

- Jira SDKs
- Google Calender API implementation classes
- GitHub API implementation classes
- SSO provider specific implementation classes

Instead adding a new integration just involves creating a new adapter instead of rewriting core business logic.

Look into using ArchUnit to fail a build if architectural boundaries are violated.

Domain Model

